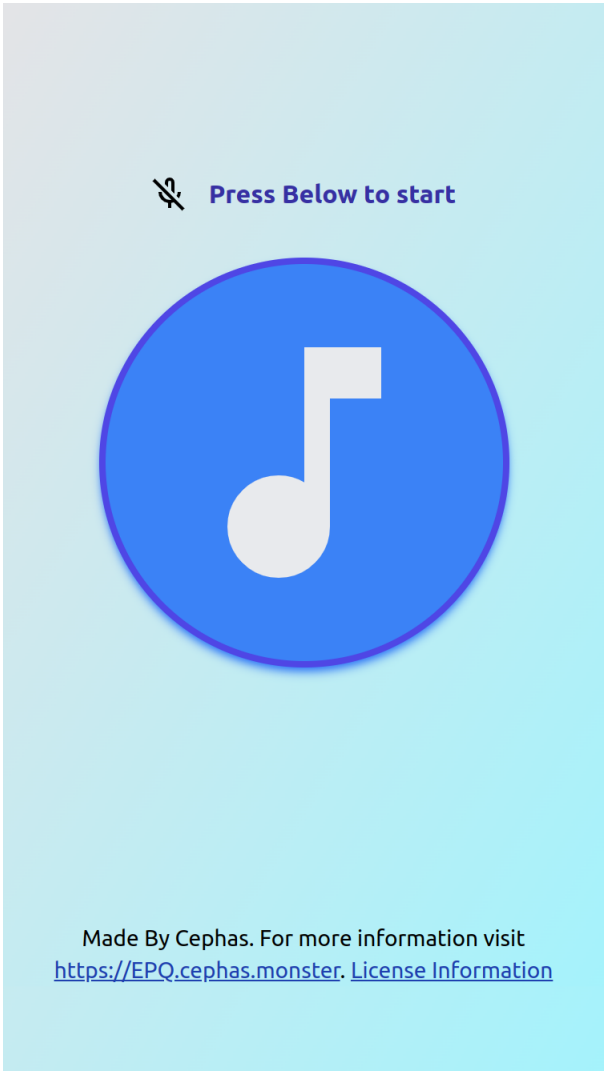
# To explore the mechanism that allows music recognition apps to work.

Suen Hang C Yeung

Candidate number: 9273

Centre number: 28278



# Contents

# Abstract

*For the purpose of this project, the replica of a music recognition app is created.*
*The algorithm first creates a fingerprint for each audio file in its spectrogram. Which are added to a database as a hashed value.*
*When record songs through the interface, the algorithm repeats the fingerprinting process, but instead compares the hash value with the matched songs.*
*During the research phase there are multiple areas that require extra research, such as testing the program and solving unexpected errors.*

# CHAPTER 1

# Introduction

In the modern digital world, millions of songs are created every single year, the demand for recognising music increases. This article answer how does modern music recognition app work.

The aim for this research is to understand and replicate how modern day music recognition algorithms work.

There are starter knowledge that are essential to know, in particular "what is Fourier Transform?". I used video tutorial from the Washington University to understand how it works.[1] However due to the complexities that it involves, I ended up using codes from (Virtanen et al. 2020) to complete the task of doing Fourier Transform.

I used a very concrete and credible source of a patent created by the original founder and Chief Scientist of Shazam®(Now part of Apple Inc®.)[2] which Shazam®is the industry leader in music recognition. The patent created by the creator of Shazam®are cited 771 times by other companies and organisation to create other resources.[3] However, due to the age of the source, some information might be irrelevant and outdated.

During the creation process, it was created using different resources to make the resource (e.g: patent, research paper) and also methods to test the success criteria set out by the algorithm.[4]

I also conducted primary research as I talked with professional. See more in section D.2.

1. Steve Brunton 2020.
2. Newnham 2023.
3. Wang and III 2013.
4. Yang 2001.

# CHAPTER 2

# Methodology / Design Choice

The software is run inside a container as the patent suggested. By containerizing the program (using docker). It allows it to be run in a "Distributed system".

The artefact is divided into two parts (Structure: Figure A.1). The web app that allows user to interact graphically. (Screenshots: Figure C.1). And the main algorithm allows the processing of the audio.

## 2.1 Web app

The web app is designed to be as user-friendly as possible.

The home page of the app would immediately present the user with the instruction text on top to remind the user to press start. And the record button are clearly outlined and highlighted with a bright colour contrast to remind the user to click on it to start the recording. Below the picture there are key text such as link to the Licensing information which. As shown in 2.1(a)

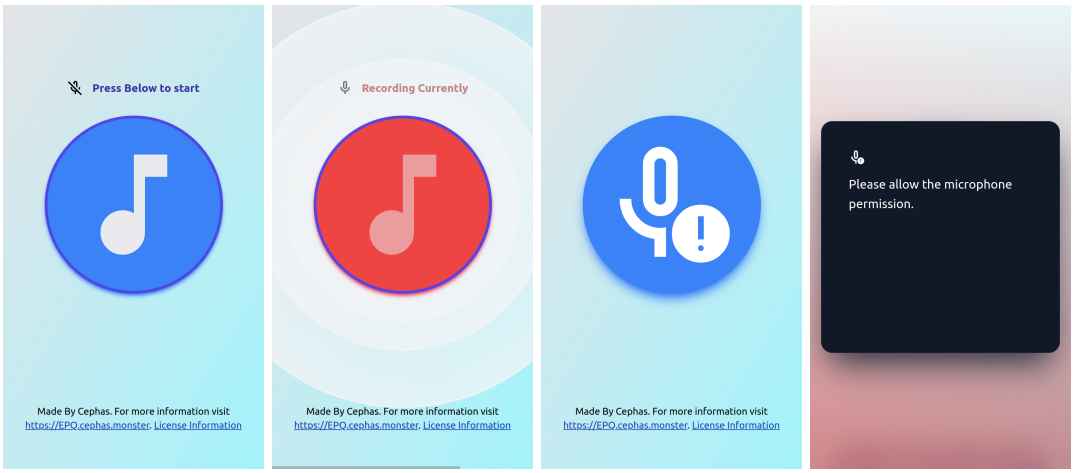
When the user start recording the music, the animated bubble effect shows up and the record button changed to red, indicating that the app is recording. As shown in 2.1(b).

If the user hasn't given the web app the permission to use microphone, the logo will show the user that they haven't given the recording permission to us. As shown in 2.1(c).

And if they disabled their microphone entirely or if they blocked the access to microphone to the website, a pop-up will show up to prevent further action from the app, as the App will no-longer works. As shown in 2.1(d).

## 2.2 The algorithm

When the program first register the song in to the database. It will go through the process of "fingerprinting" the piece of music. The algorithm consist of creating spectrogram where it uses STFT (Fast Fourier Transform dissolves a function of time or signal into the frequencies that makes




(a) (b) (c) (d)

Figure 2.1: The list of screenshot of the main screen from the web app (a) "Home", (b) Recording (c)(d) "Microphone permission are disabled"

in a way similar to how a musical chord can be expressed as the pitches of its constituent notes. It is also called the frequency domain representation of the original signals. Short-time Fourier Transform is a variation of the Fast Fourier transform built for making spectrogram[1]). Illustrated in Figure A.3.[2]

In this case, The spectrogram is being filtered through using maximum filter. The two spectrograms are compared to pick peak points. The peaks used to construct the pairs. The pairs of points are hashed using the default hashing function (Hashing function is a function where you put in any item and getting back a number. Hash functions' output tells you where the item is store and the item can be searched in a very fast time.[3]) which generate an almost unique hash values in the program. Illustrated in Figure A.4, Figure A.6 and Figure A.5.

The hash point is then placed into a database alongside with its song name and where the point A's real time. The song data (metadata) such as author and copyright detail are placed in a different database. Database example: Figure 2.2.

| Song_ID | Name | Author | Genre |
|---|---|---|---|
| f633c47a | J. S. Bach: Prelude in C - BWV 846 | Kevin MacLeod | Classical |

Figure 2.2: Example of a database entry

When the user record the sample of audio through the user interface from the web app, the recording sample is sent to the server. It is processed where it applies the same algorithm from above. First it constructs a spectrogram, peak point are picked and hash values are created, the hash value searched inside the *points* table. Same as the bottom row in Figure A.6.

1. Alpay et al. 2024.
2. Chittora and Babel 2018.
3. Bhargava 2016.

The list of successfully matched song are than going through the process of "scoring". The score calculated by constructing a histogram with the point A's real time value. (The forth column of Figure A.6) The highest score would be the correctly matched song. Using score system ensures accuracy results.[4][5][6][7]

4. Wang and III 2013.
5. MacLeod, n.d.
6. Yang 2001.
7. Wang 2003.

# CHAPTER 3

# Process of creation

## 3.1 Coding section

I started by programming the algorithm of the program, as this is the hardest part to tackle. From using the blog post source[1]. It leads me to the direction of creating the algorithm that takes in the song file and output as the database.

I initially chose to use Python to program the algorithm as Python is the most popular programming language for data analysis and other general usage.

Initially I experienced a problem where there are two different song arrays. I later found out that because most song file are represented as stereo music (where there are typically two channels eg.: left ears and right ears). So I used a music encoding program of FFmpeg to first mixed the audio back into mono audio. I cannot find how Shazam handle stereo music as stereo music are prominent at the time of the patent are published.

From the research into the method of creating spectrogram, I have narrow the research to be impossible to create a spectrogram algorithm. So I chose to produce spectrogram using programming library. Next the spectrogram are initially programmed so that it is produced using the librosa library. However, due to the constraint created by the library and the performance that the library creates, I switched and choose to produce the spectrogram from the SciPy signal library[2]. The spectrogram produce by SciPy are illustrated in Figure 3.1. (In the actual program, the spectrogram would not actually be produced because it is not necessary to produce it in the product artefact. I have created settings in the program that allows me to create diagram of how the program works.)

And the data are processed using the library of NumPy[3] through maximum filter in the library of SciPy[4]. The output of two arrays of spectrogram data are compared using NumPy technique of

1. MacLeod, n.d.
2. Virtanen et al. 2020.
3. Harris et al. 2020.
4. Virtanen et al. 2020.

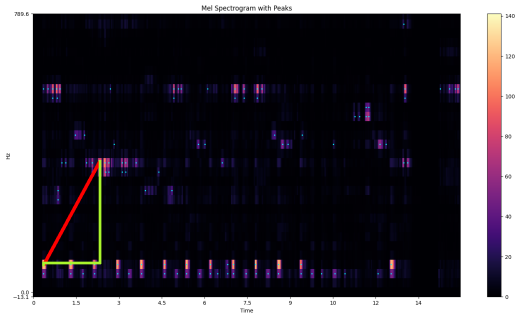



Figure 3.1: This is demonstrating how the spectrogram would look like.

boolean mask as shown in the figure in Figure A.4.

When the pairs are created from the peak picked from the maximum filter, I followed the patent to create pairs of points. However, the patent doesn't specify what is the upper and lower limit of how pair of points should be next to each other. From different testing I have concluded that the upper limit of $4000Hz$ for the frequency and $1.5second$ of time the pairs of points limited to.

In addition to the upper limits, the lower limit of $0.05second$ to prevent pairs to be too close to each other.

Having upper limit allows the program to not create too many pairs of values, as if there are no limits to how many pairs there are, there would be way too many pairs created, which reduces the uniqueness of each fingerprint profile of each song. And also reduces the performance of searching as there are more items in the database. For example: one of the song *J. S. Bach: Prelude in C - BWV 846* when there are noises added to the original sample, there are 2138 peaks, if no upper limit is imposed, there would be $2138 \times (2138 - 1) = 4568906$ pairs which is about the half the size of all the pairs stored in the actual song database for all the songs and way larger than the final with limit of 8668 pairs for the song.

Having lower limit allows points that are nearly next to each other to be omitted from pairing up, as some recording, there are imperfection in intonation or sound attenuation over distances when recording.

I also discovered from creating the program, rounding up the frequency and the time value are required for the algorithm to work properly. This is discovered during the creation phase of the project as I discussed with the professional (section D.2). This might be due to different microphone in different device records slightly differently, and their electrical design might be different. And also as sound wave propagates for some distances through the atmosphere, high-frequency components become more attenuated than low-frequency ones due to heat conduction, shear viscosity, and relaxation losses. This is noticeable after $> 15$m.[5]

5. Spiousas et al. 2017.

## 3.2 Material selection

Because this algorithm requires a library of song to demonstrate its effectiveness. I choose to use the library of song from Kevin MacLeod, all of his song library are copyright free and licensed under *the Creative Commons Attributions*. This is to avoid the copyright issue as I might infringe the *Copyright, Designs and Patents Act 1988*, section 47 as *"copying the material"*, as the algorithm are required to download the music and analysis it from it placing it inside the database.

The logo in the web app are also downloaded from the *Google Material Symbols and Icons* as they are licensed under free and open source licence of *Apache Licence 2.0*. And I think the logo from the library create a more user-friendly way of interacting the App.

## 3.3 Extra investigation

I faced challenge during the creation process, due to random noise in the sample of the audio during the fingerprinting process, I wasn't able to block the random noise in the sample, because of this when the program is sampling "Nothing" just background noise, it would appear to the user that there is song playing in the background with a random results because the program only initially took the maximum value from the list of scores. This is not mentioned in any of the articles.

I fixed this problem by using a mathematical concept of finding outliers from the list of songs. First I got all the score from the searching done inside the database. Then the interquartile range would be calculated from the list of data. If there are an outlier, that means that the matched songs are not created by random background noise from the sample itself. See Figure 3.2

There are also places where I did to avoid errors in the program such as at the start the user can record sample of the music that have a short duration, this would not be effective as the program would not be able to recognise songs. So I added an error prompt message to stop the user from uploading a song that is too short. See Figure 3.2.


Figure 3.2: Left: when duration of sample is too short. Right: when it fails to find the song

# CHAPTER 4

## Testing method

The program is tested using the method described in this paper[1] and also a meeting from professional. An evidence from all the song from (most) of the fingerprinted music collection is used to test the program, this is so that it reduce the likelihood of a flawed test.

The test have increasing level of difficulty:

- Test 1: original song file (unmodified digital file)
- Test 2: original song file but shifted
- Test 3: original song file but with controlled level of noise. (E.g.: a sine wave)
- Test 4: original song file but played over the air and recorded.
- Test 5: original song but perform by different people.
- Test 6: transposed song

The test results is inside Figure B.1.

The program overall worked as expected but test 5,6 doesn't work because this algorithm is not designed for this.

1. Yang 2001.

# CHAPTER 5

# Discussion

As discussed in the test conducted, this artefact is not capable to detect a transposed song. Because this song only matches the exact frequency that the song produces.

One way that the problem can solve is to change the mechanism entirely to not rely purely on the raw frequency data, instead by using the MIDI file of the song which contains a series of message like note.[1] Then from it, create a database of melody. Which can recognise songs from it by recording the song and passing into the algorithm to process for results. This allows a wider error range for recognizing song such as made from "humming".[2]. However, this would make the algorithm costly and slow to run.[3]

In addition, extra research might be needed to improve the speed, making it more commercially viable which is the goal set out by the Shazam®team where they designed it so that it's high performance.[4]

The research can also be further extended not limited to searching for songs, such as searching for patten inside text, audio, video, image, and any multimedia combinations of individual media types. For example: image fingerprints can be pixel RGB values.[5]

In addition, this project only uses copyright-free music, as there would be legal and ethical implication to using contemporary music.

The biggest ethical implication that most music recognition app faced are privacy violation, as some app record the audio in the background even without user interaction. The project would not be concerned about this because it only records when user intended to.

1. amandaghassaei, n.d.
2. Ghias et al. 1995.
3. Yang 2001.
4. Wang and III 2013.
5. Wang and III 2013.

# CHAPTER 6

# Conclusion

Further research might be needed into improving the accuracy, making it able to recognise variation of the same song.[1]

In conclusion, It met the expectation of the project end goal which are tested using a vigorous testing method set out by a professional opinion and an article.

1. Yang 2001.

# Bibliography


Alpay, Daniel, Antonino De Martino, Kamal Diki, and Daniele C. Struppa. 2024. Short-time fourier transform and superoscillations. *Applied and Computational Harmonic Analysis* 73 (November 1, 2024): 101689. ISSN: 1063-5203, accessed March 9, 2025. https://doi.org/10.1016/j.acha.2024.101689. https://www.sciencedirect.com/science/article/pii/S1063520324000666.

amandaghassaei. n.d. What is MIDI? Instructables. Accessed January 31, 2025. https://www.instructables.com/What-is-MIDI/.

Bhargava, Aditya Y. 2016. *Grokking algorithms: an illustrated guide for programmers and other curious people.* Manning Publications. ISBN: 978-1-61729-223-1.

Chittora, Nisha, and Deepika Babel. 2018. A BRIEF STUDY ON FOURIER TRANSFORM AND ITS APPLICATIONS. 05 (12).

Ghias, Asif, Jonathan Logan, David Chamberlin, and Brian C. Smith. 1995. Query by humming: musical information retrieval in an audio database. In *Proceedings of the third ACM international conference on multimedia - MULTIMEDIA '95,* 231–236. the third ACM international conference. San Francisco, California, United States: ACM Press. ISBN: 978-0-89791-751-3, accessed January 31, 2025. https://doi.org/10.1145/217279.215273. http://portal.acm.org/citation.cfm?doid=217279.215273.

Harris, Charles R., K. Jarrod Millman, Stéfan J. Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. Array programming with NumPy. *Nature* 585, no. 7825 (September 17, 2020): 357–362. ISSN: 0028-0836, 1476-4687, accessed January 12, 2025. https://doi.org/10.1038/s41586-020-2649-2. https://www.nature.com/articles/s41586-020-2649-2.

MacLeod, Cameron. n.d. Abracadabra: how does shazam work? - cameron MacLeod. Accessed October 28, 2024. https://www.cameronmacleod.com/blog/how-does-shazam-work.

Newnham, Danielle. 2023. INTERVIEW: dr avery wang, principal research scientist at apple and co-founder and chief scientist... Medium, February 2, 2023. Accessed February 2, 2025. https://daniellenewnham.medium.com/fostering-innovation-and-choosing-the-right-founding-team-6287f7eed9f.

Spiousas, Ignacio, Pablo E. Etchemendy, Manuel C. Eguia, Esteban R. Calcagno, Ezequiel Abregú, and Ramiro O. Vergara. 2017. Sound spectrum influences auditory distance perception of sound sources located in a room environment. *Front Psychol* 8 (June 22, 2017): 969. ISSN: 1664-1078, accessed March 9, 2025. https://doi.org/10.3389/fpsyg.2017.00969. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5479918/.

Steve Brunton. 2020. *The fast fourier transform algorithm.* April 4, 2020. Accessed October 29, 2024. https://www.youtube.com/watch?v=toj_IoCQE-4.

Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in python. Publisher: Nature Publishing Group, *Nat Methods* 17, no. 3 (March): 261–272. ISSN: 1548-7105, accessed October 28, 2024. https://doi.org/10.1038/s41592-019-0686-2. https://www.nature.com/articles/s41592-019-0686-2.

Wang, Avery Li-Chun. 2003. An industrial-strength audio search algorithm (January).

Wang, Avery Li-Chun, and Julius O. Smith III (Shazam Investments Ltd). 2013. Systems and methods for recognizing sound and music signals in high noise and distortion. U.S. patent 8386258B2, filed February 26, 2013. Accessed December 25, 2024. https://patents.google.com/patent/US8386258B2/en?assignee=Shazam+Investments+Ltd&num=25.

Yang, Cheng. 2001. *Music database retrieval based on spectral similarity.* Technical Report 2001-14. Backup Publisher: Stanford InfoLab. Stanford. http://ilpubs.stanford.edu:8090/489/.

# APPENDIX A

## Figures

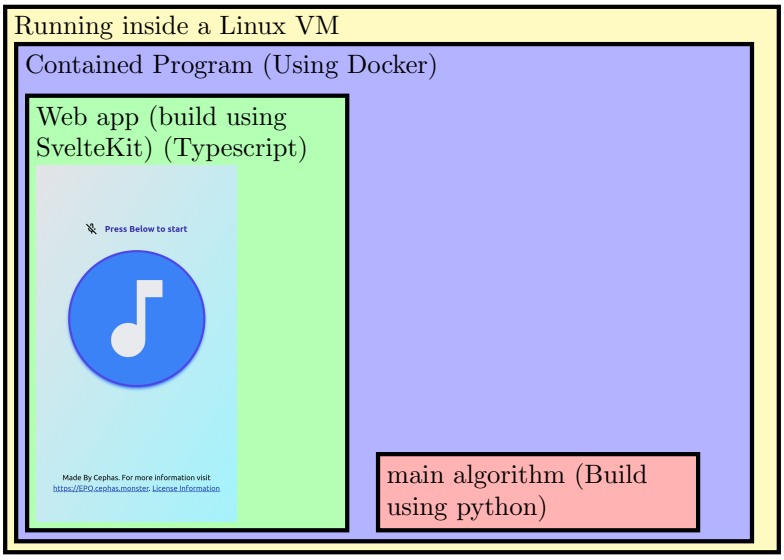



Figure A.1: Overall Architecture

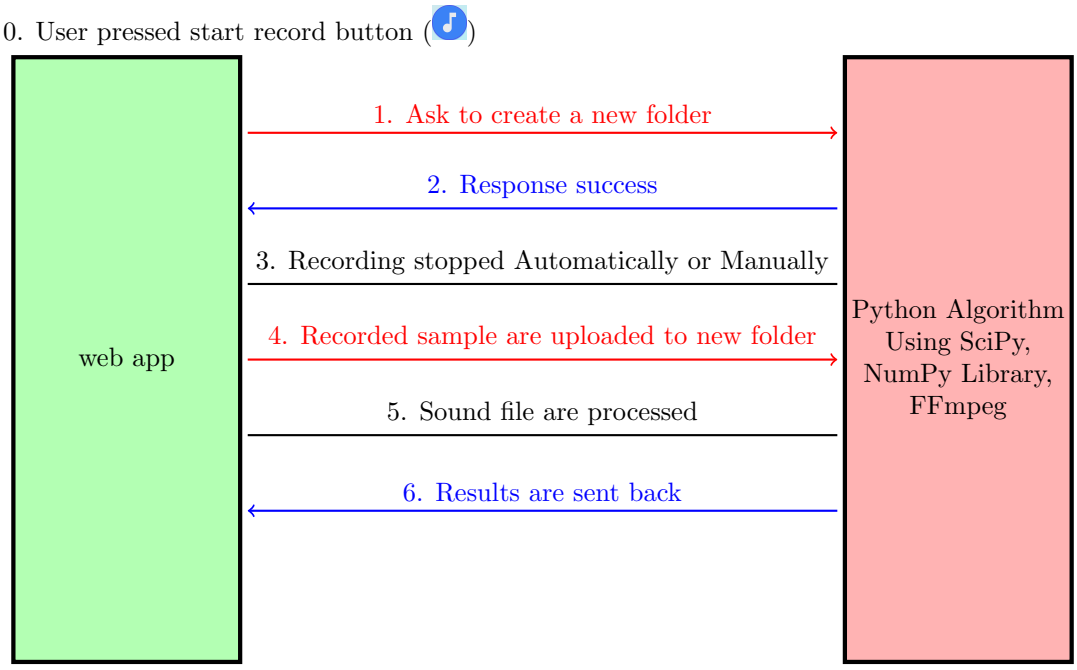



Figure A.2: Overall Architecture (app)

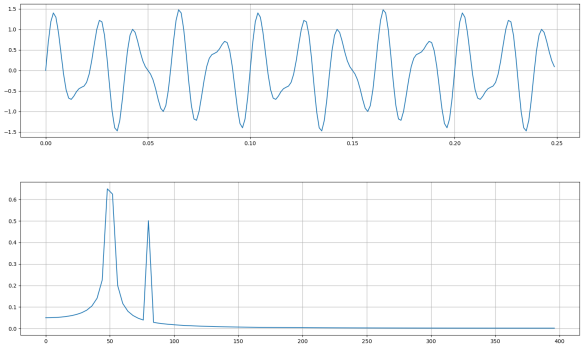


Figure A.3: Fast Fourier Transform as an example through transforming the top signal in to representation in the frequency domain at the bottom.

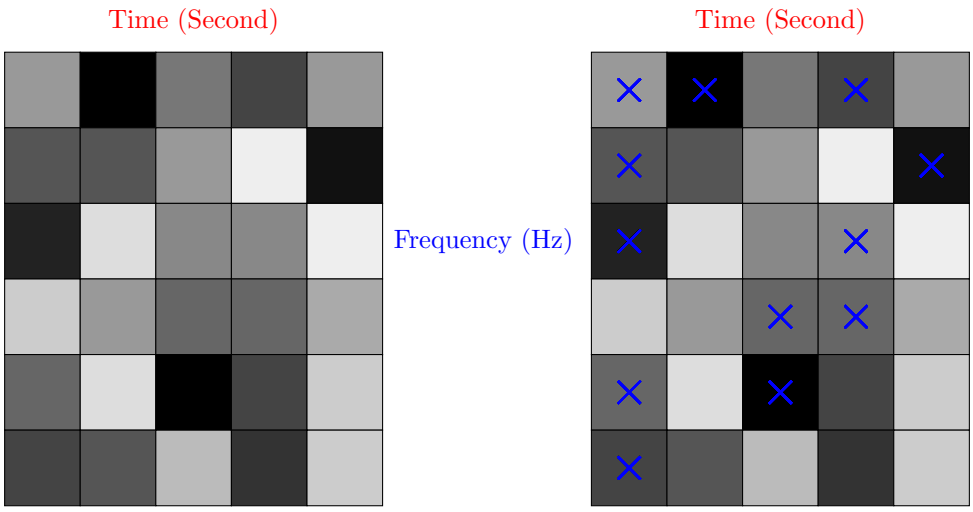



Figure A.4: Illustrating an example of a spectrogram picking of points. The darker the box, the higher the amplitude (value) represents

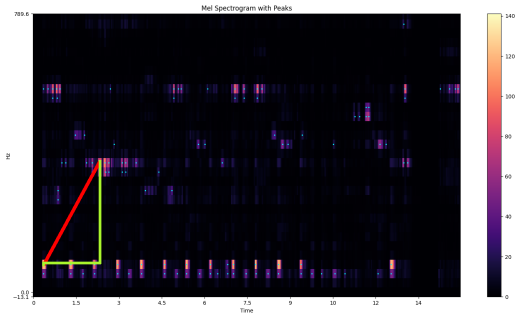



Figure A.5: The tip of the triangle represents Point A and Point B. Y-axis represents the time, X-axis represents the frequency, The amplitude scales are represented by the colour bar on the right.

| Point A (Hz) | Point B (Hz) | Time delta (s) | Point A time (s) | Track UUID |
|---|---|---|---|---|
| 20.1 | 302.1 | 1.8 | 0.3 | *ea47a3c5* |
| 2388532207457024332 | | | 2.1 | *ea47a3c5* |

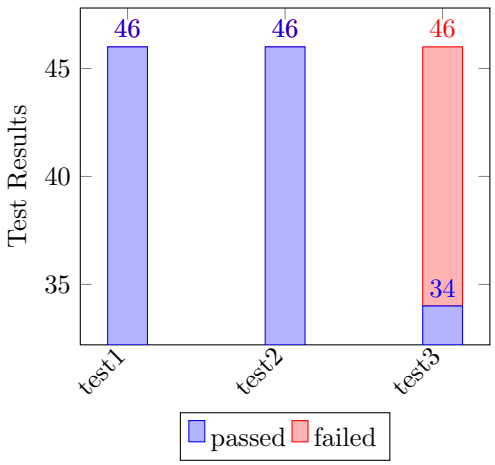
Figure A.6: Demonstrate / example of how item are hashed. MacLeod, n.d.

| Song_ID | Name | Author | Genre |
|---|---|---|---|
| f633c47a | J. S. Bach: Prelude in C - BWV 846 | Kevin MacLeod | Classical |

Figure A.7: Example of a database entry

# APPENDIX B

## Testing results



Test 1: (tested on 100% of the song library)

| filepath | passed | SUBTOTAL |
|---|---|---|
| test/test_1.py | 46 | 46 |
| TOTAL | 46 | 46 |

Test 2: (tested on 100% of the song library)

| filepath | passed | SUBTOTAL |
|---|---|---|
| test/test_2.py | 46 | 46 |
| TOTAL | 46 | 46 |

Test 3: (tested on 100% of the song library)

| filepath | passed | failed | SUBTOTAL |
|---|---|---|---|
| test/test_3.py | 34 | 12 | 46 |
| TOTAL | 34 | 12 | 46 |

Test 4: Most works (Only tested 40% of the song library)

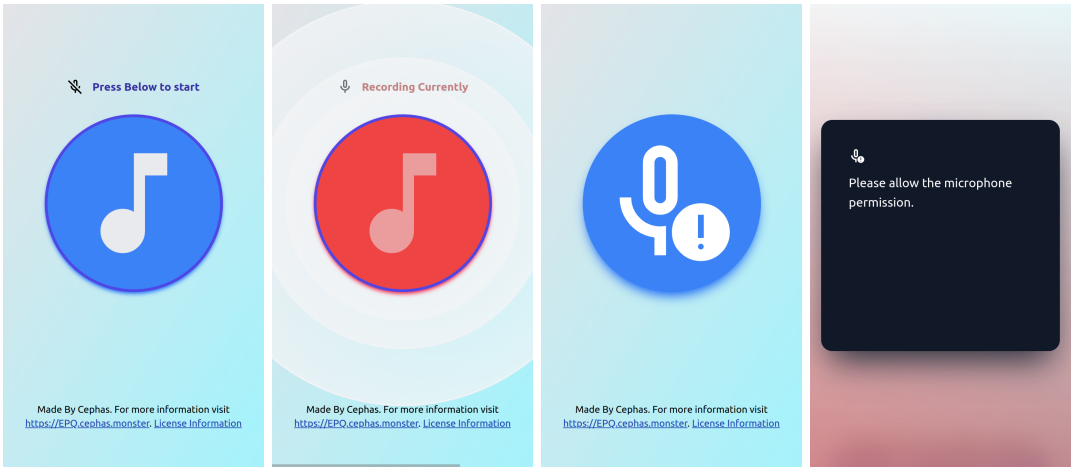
Test 5: Most failed (Only tested 20% of the song library)

Test 6: All failed (Only tested 10% of the song library)

Figure B.1: test result table

# APPENDIX C

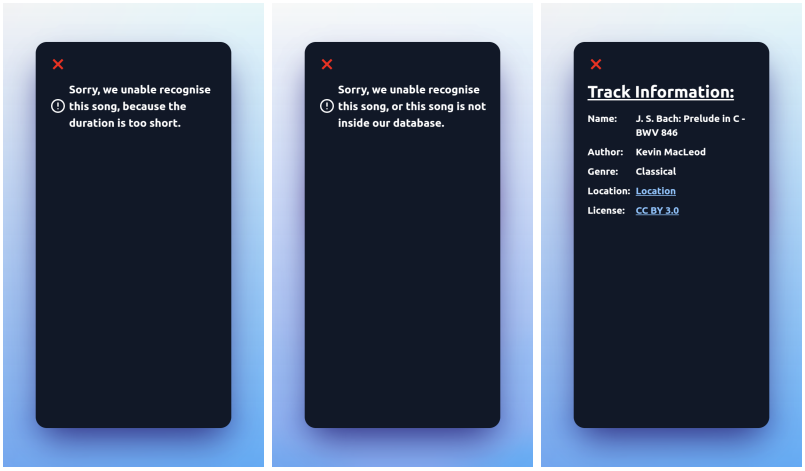
## Screenshots




(a) (b) (c) (d)

Figure C.1: (a) "Home", (b) Recording (c)(d) "Microphone permission are disabled"

All the figures are simulated to the size of an iPhone®SE display.




(a) (b) (c)

Figure C.2: (a) "Too Short", (b) "No song", (c) "Recognized song (e.g: Prelude by Bach)"

# APPENDIX D

# Meeting logs

## D.1 Meeting with the supervisor at 25 November 2024

Summary:

Improve on the part A of the production logs, with more details on the titles described and what resources I will be using (such as the URLs should be provided), the timescale and what is involve.

As things I should be including to create a great project is the benefit that this might bring to the society and the impact.

And a plan to see what I should be sending or showing to the examiner in the future such as:

- Send product log
- Presentation of the artefact
- Evidence of the artefact
- Critical bibliography
- Place that in the Appendix and write a reference back to the appendix through the production log.

## D.2 Meeting with a professional at 13 January 2025

Summary:

I explained the background details during the meeting.

I have explained that the program can only detect sounds from following test that I conducted such as:

- A test where there are no modifications to the original sound,
- A test where the song has cropped and reduced to a smaller size.
- A test where there is a constant noise generated by a program with a constant sine wave in the background.
- A test where the songs are mixed with a normal acceptable background noise such as coughing.

All of those above works as expected.

However, the test that I did by recording when the sound is played on a music player on a different device doesn't work.

As an additional note, he has mentioned that the program is slow because of a reason which might be caused by the hash point not being an integer, as the hash value using integer is faster than using string as hash. (It was diagnosed with a slow database reading speed prior to the meeting).

The project has concluded as partially successful from the results produced from the searching algorithm.

**Summary of Improvement I need to make from the project.**

I should use a different hashing algorithm for the fingerprinting part of the program.

Round up the frequency, and the changes in time that are fed into the hashing algorithm.

**Additional note:**

The meetings are conducted with the consent of the professional. (in regards to the ethical issues)

The professional is my computer science teacher, as he has a decent qualification of a university degree.

# APPENDIX E

## Critical Bibliography

| Sources | | Critics | Notes |
|---|---|---|---|
| Title: Short-time fourier transform and superoscillations<br>Author: Alpay et al.<br>Issue Date: November 1, 2024<br>URL: https://www.sciencedirect.com/science/article/pii/S1063520324000666 | Alpay et al. 2024 | This article is indented to provide information about the basics of Short-time Fourier Transform. | |
| Title: What is MIDI?<br>Author: amandaghassaei<br>Issue Date:<br>URL: https://www.instructables.com/What-is-MIDI/ | amandaghassaei, n.d. | This tutorial article is indented to provide a really basic definition about the MIDI file. | |
| Title: Query by humming<br>Author: Ghias et al.<br>Issue Date: 1995<br>URL: http://portal.acm.org/citation.cfm?doid=217279.215273 | Ghias et al. 1995 | This conference article provide a discussion point on how to make the artefact if I want to do it differently. This source is credible to an extent as it's written by ACM which is a scientific and educational computing society. However, it might be slightly outdated (published date 1995) as there might be more information about this. | |
| Title: Array programming with NumPy<br>Author: Harris et al.<br>Issue Date: September 17, 2020<br>URL: https://www.nature.com/articles/s41586-020-2649-2 | Harris et al. 2020 | This program is indented to be used in the program. This program extensively used in data analysis. It's an industry standard tool to analysis array in all fields, such as research. | Utilized only in the code |

| | | | |
|---|---|---|---|
| Title: Abracadabra<br>Author: MacLeod<br>Issue Date:<br>URL: https://www.cameronmacleod.com/blog/how-does-shazam-work | MacLeod, n.d. | This blog post is important because this includes much more details that the original article from the Shazam company. However, this does not mean this blog post are accurate as this is from written from a perspective of the original creator. But the writer is partially qualified to write about this topic as the writer work as a software engineer at Bloomberg and a graduate of University of Edinburgh. | |
| Title: INTERVIEW<br>Author: Newnham<br>Issue Date: February 2, 2023<br>URL: https://daniellenewnham.medium.com/fostering-innovation-and-choosing-the-right-founding-team-6287f7eed9f | Newnham 2023 | This interview give an insight into the creator of Shazam Avery Wang's identity, which his source is used as concrete source in the program. Wang and III 2013; Wang 2003 | |
| Title: SciPy 1.0<br>Author: Virtanen et al.<br>Issue Date: March 2020<br>URL: https://www.nature.com/articles/s41592-019-0686-2 | Virtanen et al. 2020 | This program is indented to be used in the program. This program extensively used in solving mathematical problem and scientific tool. And it's an industry standard programming interface for solving mathematical problem and analysing scientific problem. | Utilized only in the code |

| | | | |
|---|---|---|---|
| Title: An industrial-strength audio search algorithm<br>Author: Wang<br>Issue Date: January 2003<br>URL: | Wang 2003 | This is the original article explaining from a prospective of the creator. This article creates a basic understanding of how the basics works. They have a detail analysis of recognition rate and the performance associated. However, the article is missing from explaining the method of how they create Spectrogram, and the algorithm they are using to create it. There might be changes to the modern day algorithm that the product is using, this might be due to the age of this paper being published in 2003. And since now the company are acquired by Apple Inc. | |
| Title: Systems and methods for recognizing sound and music signals in high noise and distortion<br>Author: Wang and III<br>Issue Date: February 26, 2013<br>URL: https://patents.google.com/patent/US8386258B2/en?assignee=Shazam+Investments+Ltd&num=25 | Wang and III 2013 | This source is very credible and concrete, as the original program Shazam are based on it. It is created by the founder and principle scientist of Shazam. Inside the source there are also figures attached to it. This patent is cited (as of February 2025) 771 times. It's being used in various different field, most notability social media companies and big tech companies such as Google, Snap. However, the source might not be as relevant because the patent are outdated and expired. | |
| Title: *Music database retrieval based on spectral similarity*<br>Author: Yang<br>Issue Date: 2001<br>URL: http://ilpubs.stanford.edu:8090/489/ | Yang 2001 | This source provides resources mainly how to test the artefact. This report are also written by the Stanford University which is a very credible source and provides great information. | |

# APPENDIX F

## Timetable

See the landscape table file after this page.

- Green done means on time
- Orange done means not on time

I choose to use timetable target set by week because it allows for more streamline approach to time management.

timetable

| Todos | Expected Length | Recommanded Date | Final Length | Final Date | Done | Notes |
|---|---|---|---|---|---|---|
| learning about how spectrogram are made | 1 week | 27/10/2024 | 1 week | 5/11/2024 | ✓ | / |
| Make a spectrogram | 1 week | 4/11/2024 | 2 week | 20/11/2024 | ✓ | The creation process takes longer than expected, as I have to figure out out of memory issues. |
| make a contellation map of it | 2 week | 18/11/2024 | 1 week | 1/12/2024 | ✓ | |
| store the data point as a hash point | 2 week | 2/12/2024 | 2 week | 2/12/2024 | ✓ | The Time it takes to complete it are shorter than expected |
| place the hash point in a database | 2 week | 16/12/2024 | 3 week | 25/12/2024 | ✓ | The process took longer than expected due to unexpected event happening outside of the EPQ. |
| make a web app that allows recording of the music | 2 week | 16/1/2025 | 2 week | 30/1/2025 | ✓ | |
| linking everything together | 2 week | 1/2/2025 | 3 week | 6/2/2025 | ✓ | Both process are interlinked to each other and the time of completion are linked |
| testing process | 2 week | 13/2/2025 | 1 week | 13/2/2025 | ✓ | |
| write a report about it | 3 week | | | | | Both process are interlinked to each other as I am required to test the program while writing the report |
| make slideshow to present | 2 week | | | | | |

# APPENDIX G

## Links

| | |
|---|---|
| The Artefact: https://recognizer.cephas.monster/ |  |
| EPQ directory: https://EPQ.cephas.monster/ |  |
| Privacy statement: https://EPQ.cephas.monster/privacy |  |

| | |
|---|---|
| EPQ code: https://EPQ.cephas.monster/code | |
| This report: https://EPQ.cephas.monster/report | |